

Ficha da Etapa 1 — Projeto de Solução Distribuída

Disciplina: 070080 Sistemas Distribuídos e Paralelos · Oficina: 08/09 · Turma: CC6MA · Professor: Fábio Rocha de Araújo · Etapa 1 completa: 22/09

A. Equipe e problema

#	Nome	Matrícula
1	Yuri Aguiar	24070309
2	Pedro Paulo	24070313
3	João Rath	24070338
4	Fábio Gabriel	21070209

Problema escolhido: triagem automática de editais de compra pública. A aplicação lê um lote de editais em texto, pontua cada um contra uma taxonomia de termos ligados a software de gestão e arrecadação pública, classifica por linha de produto e calcula uma assinatura MinHash do objeto para identificar o mesmo edital republicado por municípios diferentes. A saída é um relatório com ranking, distribuição por linha, frequência de termos e contagem de republicações.

Condição	Resposta da equipe
O trabalho se divide em partes independentes? Em que unidade?	Sim. A unidade é um edital . O processamento de um edital não lê nem escreve nada de outro: as funções de <code>src/nucLeo.py</code> são puras e o módulo não tem estado global. A dependência só aparece na agregação dos resultados, e a agregação é associativa e comutativa.
Qual é o volume da entrada, e quanto tempo a versão sequencial deve levar?	8000 editais de cerca de 10 KiB cada, 81 MiB de texto. O volume não foi arbitrado: <code>src/calibrar.py</code> mede o custo por documento na própria instância e calcula quantos documentos são precisos para o alvo de 3 minutos de execução sequencial. Medido na instância: 2,9 min .
Como se verifica que o resultado está correto?	O relatório final é canonizado — a ordenação depende só do conteúdo, nunca da ordem de chegada — e reduzido a um SHA-256 . <code>src/verificar.py</code> roda a versão sequencial uma vez e a paralela várias vezes sobre a mesma entrada e compara as impressões digitais. São duas verificações distintas: <i>equivalência</i> (paralela = sequencial) e <i>estabilidade</i> (todas as execuções paralelas iguais entre si).
Que estado é escrito por mais de um fluxo?	O agregador global, em memória compartilhada por um <code>multiprocessing.Manager</code> : contador de documentos e de tokens, soma de pontuação, dicionário de frequência dos termos da taxonomia, contagem por linha de produto, ranking dos 25 melhores, mapa de assinaturas e relógio lógico.

B. Estratégia de paralelização

Decisão	Escolha	Justificativa
Paralelismo de dados ou de tarefas	Dados	A mesma sequência de operações se aplica a documentos diferentes. Não existem estágios heterogêneos que pudessem correr simultaneamente, então paralelismo de tarefas não se aplica.
Processo ou thread	Processo	O trabalho é limitado por processador , não por espera: a leitura do arquivo responde por menos de 0,5 % do tempo total e o restante é normalização Unicode, expressão regular, hashing e aritmética de inteiros em Python puro. Em CPython o GIL impede que duas threads executem bytecode ao mesmo tempo, de modo que a versão com threads mediria speedup próximo de 1 e a análise concluiria o oposto do correto. A aplicação tem o modo <code>--motor thread</code> para que essa afirmação seja medida, e não apenas citada.
Quantos trabalhadores em paralelo	1 e 2	A instância <code>c5.large</code> tem 2 vCPU sobre 1 núcleo(s) físico(s). Mede-se a curva inteira para localizar onde o ganho satura. Com um único núcleo físico, os 2 vCPU são threads de hardware do mesmo núcleo: o segundo trabalhador não dobra a capacidade de cálculo, e o speedup medido deve ficar bem abaixo do teto de Amdahl. A medição existe justamente para mostrar o tamanho dessa diferença. O teto de Amdahl para $n = 2$ limita o ganho a 2,00, e esse teto é propriedade da máquina, não do programa.
Como o trabalho é dividido entre eles	Fila comum de lotes	O corpus é cortado em lotes de 150 documentos, todos postos numa única multiprocessing.Queue . Cada trabalhador puxa o próximo lote quando termina o anterior: balanceamento dinâmico. Divisão fixa em n fatias deixaria o tempo total refem da fatia mais cara.

C. Sincronização

Item	Resposta da equipe
Que estrutura é compartilhada e escrita por mais de um fluxo	O agregador global (<code>estado</code> , assinaturas e eventos, todos proxies de <code>Manager</code>), escrito por todos os trabalhadores ao fim de cada lote.
Qual é o menor trecho que precisa ser indivisível (a seção crítica)	A função <code>fundir_no_global</code> , em <code>src/paralelo.py</code> : a fusão de um agregado parcial no agregado global. O processamento dos 150 documentos do lote acontece fora da trava, num parcial local ao processo. Dentro da trava ocorrem duas operações que não toleram entrelaçamento: a leitura-modificação-escrita dos contadores (<code>estado["documentos"] = estado["documentos"] + k</code>) e o verificar-entao-agir sobre o mapa de assinaturas.
Que primitiva vai protegê-la: lock, semáforo ou monitor	multiprocessing.Lock . O recurso admite um escritor por vez e não há condição de espera a sinalizar nem contagem de permissões a administrar, o que dispensa semáforo contado e monitor. A trava é liberada em <code>finally</code> , para que uma exceção dentro da seção crítica não trave os demais processos.
Como a equipe vai demonstrar que o resultado ficou estável	Ao vivo, com <code>src/verificar.py</code> : a mesma entrada, oito execuções paralelas, oito SHA-256 idênticos e iguais ao da versão sequencial. Em seguida a mesma verificação com <code>--modo nenhuma</code> , que remove a trava: aí aparecem impressões digitais diferentes a cada execução e contagens de documentos menores que o total, porque atualizações se perdem. A corrida é exibida, não descrita.

Registro de evento e carimbo lógico. Há troca de mensagem: a fila de lotes e o canal até o processo do `Manager`. Como os processos não compartilham relógio físico, a ordem das fusões é registrada por um **relógio lógico de Lamport**: cada trabalhador mantém um contador local, incrementa-o ao receber um lote e, ao fundir, adianta-o para `max(local, global) + 1`, grava esse valor no estado compartilhado e registra o evento. Cada registro contém:

Campo	Conteúdo
quem	identificador do trabalhador (<code>w0</code> a <code>w1</code>)
o quê	<code>fusao_no_agregador_global</code>
carimbo lógico	valor do relógio de Lamport após o avanço
sobre o quê	número do lote e quantidade de documentos fundidos

O registro completo é gravado em `resultados/eventos.json`, ordenado por carimbo, e permite reconstruir uma ordenação consistente das fusões sem relógio comum.

D. Medição de desempenho

Item	Resposta da equipe
Máquina em que as duas versões serão medidas	Instância única <code>c5.large</code> (AWS EC2), região <code>us-east-1</code> (Norte da Virgínia), zona <code>us-east-1c</code> . As duas versões rodam na mesma instância, na mesma sessão, sem outra carga.
Número de núcleos disponíveis nela	2 vCPU sobre 1 núcleos físicos (SMT ativo). A distinção importa: a lei de Amdahl supõe <i>n</i> processadores independentes, e dois vCPU sobre o mesmo núcleo físico não são independentes para trabalho limitado por processador.
Volume de entrada fixado para as medições	8000 editais, gerados por <code>src/corpus.py</code> com semente fixa. A mesma semente reproduz byte a byte o mesmo corpus, o que garante que $T(1)$ e $T(n)$ sejam medidos sobre a mesma entrada.
Como o tempo sequencial será obtido	Executando <code>src/sequencial.py</code> , que não tem fila, <code>Manager</code> nem trava, três vezes, tomando a mediana. Não se usa a versão paralela com um trabalhador como $T(1)$: ela já carrega o custo da infraestrutura de paralelização e inflaria o speedup.
Fração do trabalho que a equipe estima ser paralelizável	$p \approx 0,9977$ (medido por instrumentação). Fica fora do trecho paralelo: a listagem do diretório, a partida dos processos, a redução final e a gravação do relatório. A versão sequencial cronometra separadamente a leitura e o trabalho por documento, de modo que o valor instrumentado entra no relatório de 22/09 junto com a fração serial experimental de Karp-Flatt, $e = (1/S - 1/n) / (1 - 1/n)$.
Speedup previsto pela lei de Amdahl	$S = 1 / ((1 - p) + p/n)$, com $p = 0,9977$: $n = 2 \rightarrow 2,00$. A equipe espera ficar <i>abaixo</i> desses tetos, e o relatório vai explicar a diferença pelas três causas que a lei não modela: comunicação entre processos, espera na seção crítica e vCPU que não são núcleos independentes.

E. Arquitetura na nuvem

Decisão	Escolha da equipe	Justificativa
Região, e em quantas zonas de disponibilidade	<code>us-east-1</code> (Norte da Virgínia) uma zona (<code>us-east-1c</code>)	Instância isolada em zona única: passa a valer o compromisso de 99,5 % (216 min/mes) . Distribuir por duas ou mais zonas elevaria o compromisso para 99,99 % (4,3 min/mes), mas o experimento exige que $T(1)$ e $T(n)$ sejam medidos no mesmo hardware — duas instâncias em zonas diferentes não comparariam a mesma coisa — e a aplicação é um lote sob demanda, sem requisito de disponibilidade contínua. A região não foi uma escolha de projeto: a conta disponível é um AWS Academy Learner Lab, que restringe a operação a <code>us-east-1</code> . O experimento não depende disso — $T(1)$ e $T(n)$ são medidos na mesma instância, e a latência até a equipe não entra no tempo medido, porque o corpus é lido do disco local.
Família, tamanho e quantidade de instâncias	$1 \times$ <code>c5.large</code> (2 vCPU, otimizada para computação)	Família sem créditos de CPU. Instância burstável (t_2 , t_3) é inadequada para medir speedup: ao esgotar os créditos o desempenho cai no meio da medição e $T(n)$ deixa de ser comparável a $T(1)$. Uma só instância, porque o objeto da medida é o ganho por núcleo dentro de uma máquina.
Onde ficam a entrada e a saída, e qual o volume	EBS gp3 de 30 GiB; saída versionada no repositório	O corpus (81 MiB) é gerado no volume local, para que a leitura não introduza latência de rede dentro do tempo medido. A saída são <code>medicao.json</code> e <code>eventos.json</code> , dois arquivos de poucos KiB, versionados no repositório junto com o código que os gerou — assim sobrevivem ao encerramento da instância e ficam na mesma versão do código. Um bucket S3 resolveria o mesmo problema com um serviço a mais para provisionar e controlar; para este volume não se justifica.
Portas abertas, e a origem de cada regra	<code>22/tcp</code> ← <code>177.180.12.98/32</code> <code>8000/tcp</code> ← <code>0.0.0.0/0</code> saída: liberada	A porta administrativa não está aberta para <code>0.0.0.0/0</code> porque a porta 22 exposta à internet recebe tentativas automáticas de autenticação de forma contínua; restringir a origem ao endereço da equipe retira a instância dessa superfície sem depender da força da chave. A porta 8000 serve o painel da aplicação e é a única aberta ao público. A saída fica liberada para instalar pacotes e enviar os resultados ao S3.
A instância fica ligada, ou é criada por execução	Ligada durante a janela de medição e apresentação	Criar por execução traria hardware subjacente potencialmente diferente entre $T(1)$ e $T(n)$, e o speedup exige a mesma máquina. A instância é encerrada após a apresentação.